# Due: April 12, 2026, 11:59 PM | 100 Points

Instructor: Junaid Rashid

**Week 5**

## Collaboration Policy

This is an **individual assignment**. You may discuss concepts with classmates, but all code must be written by you. Sharing or copying code results in zero marks for all parties involved.

## Setup

Use Google Colab (recommended) or a local Python environment (Anaconda/Spyder, VS Code, PyCharm).

## Introduction

You and a group of friends are launching your own open source software project. You are the **Project Lead**. Build a **Contributor Management and Issue Tracking System** in Python. The project, contributors, and issues are decided by you that collect all data using **input()**. Write **one program** that runs top to bottom. Data from early sections must be used in later ones.

### Section 1 — Launch Your Project (25 points)

Print a welcome banner using **string repetition** (no manual typing). Store official project info in a **Tuple** called **project**:

- Fields: name, version, year started, main language, your name as lead
- Print each field using its **index**
- Use **slicing** to print the first three fields at once
- Use **count()** and **index()** on the tuple
- Add a **comment** showing what happens if you try to change a tuple value, explain why tuples suit this data.

Use **input()** to register **exactly 4 contributors** (name, role, language, commits, country). Store each as a **dict**, append to **contributors** list. Then:

- Extract names into a **names** list → sort → print → negative index → slice first two
- Loop through contributors and use **update()** to add **'status': 'Active'** to each
- Use **get()** to print the first contributor's status
- Use **copy()** to back up the first contributor's dict and print it

**Expected Output (your data will differ):**

```
==========================================
   OpenTrack — Open Source Project
   Project Lead : Ali Hassan
==========================================
```

```
Name : OpenTrack  |  Version : 1.0  |  Started : 2024
First 3 fields  : ('OpenTrack', '1.0', 2024)
Language count  : 1   Language index : 3
# project[0] = 'test' → TypeError: tuple does not support item assignment
----------------------------------------
Sorted names    : ['Fatima', 'Jisoo', 'Minho', 'Sara']
Last name       : Sara       First two : ['Fatima', 'Jisoo']
Contributor 1 status : Active
Backup : {'name': 'Minho', 'role': 'Backend Dev', ..., 'status': 'Active'}
```

## Section 2 — Track and Analyse Issues (30 points)

Use **input()** to collect **exactly 5 issues** (id, title, type: Bug/Feature, priority: Critical/High/Medium/Low, reporter, status: Open/In Progress/Resolved). Store each as a dict in a list called **issues**. Then:

**List operations:**

- Count **'Open'** issues using a loop and counter — **do not use count()**
- Change the **first issue's** priority to **'Critical'** using its index
- Use slicing to print the **last two** issues

**Set operations:**

- Build **reporters** set from issues (loop) and **tech_stack** set from contributors (loop)
- **add()** one more technology manually — **discard()** one that may or may not exist
- **union()** both sets — **intersection()** to find common items — **difference()** to find reporters not in tech_stack
- Check if **'Critical'** is in a set of all priorities — print a clear message

**Dictionary operations:**

- Build **priority_count** dict by looping — map each priority to its count (no hardcoding)
- Build **status_groups** dict — each key is a status, each value is a list of issue titles
- Use **keys()**, **values()**, and **items()** each at least once when printing
- Find the top reporter using a loop and dict — **do not use max() or Counter()**
- Use **pop()** to remove the **'type'** key from the first issue and print the result

**Expected Output (partial — your data will differ):**

```
Open issues       : 3
First issue → priority updated to Critical.
Last two issues   : [{'id': 'ISS-004', ...}, {'id': 'ISS-005', ...}]
----------------------------------------
reporters         : {'Minho', 'Sara', 'Fatima', 'Jisoo'}
tech_stack        : {'Python', 'JavaScript', 'Rust', 'TypeScript'}
union             : {'Minho', 'Sara', 'Python', ...}
intersection      : set()       difference : {'Minho', 'Sara', ...}
Critical present  : YES — flag for immediate review.
----------------------------------------
priority keys     : dict_keys(['Critical', 'High', 'Medium'])
priority values   : dict_values([2, 2, 1])
Status Groups → Open : ['Login crash', 'Dark mode'] | Resolved : ['README typo']
Top reporter      : Minho (2 issues)
After pop('type') : {'id': 'ISS-001', 'title': ..., 'priority': 'Critical', ...}
```

## Section 3 — Save, Read and Report (35 points)

**Folder & files:**

- Create a folder named after your project (from tuple, lowercase, underscores)
- Check with **os.path.exists()** before creating — skip if it already exists
- Use **os.path.join()** for all paths — print folder contents with **os.listdir()**
- Save **project_report.txt** — full report (project info, contributors, issues, analysis, top reporter)
- Save **issues.csv** — header: **id,title,priority,reporter,status** then one row per issue
- Wrap all file writing in **try/except IOError**

**Reading back:**

- **read()** — print full file content
- **readline()** — print first two lines only
- **readlines()** — count total lines, print only lines containing **'Critical'** or **'High'**
- Wrap reading in **try/except FileNotFoundError**

**Final printed summary:**

- Use **only f-strings** — no **+** concatenation anywhere in this section
- Every value from a variable — no hardcoding
- Include: project name/version (from tuple), contributors, sorted names, total/open issues, reporters, tech stack, top reporter, priority breakdown, file paths saved

**Expected Output (partial — your data will differ):**

```
Folder created : open_track/
Files saved    : ['issues.csv', 'project_report.txt']
--- read() ---         [full file content prints here]
--- readline() ---
Line 1 : ==========================================
Line 2 :    OpenTrack — PROJECT REPORT
--- readlines() ---
Total lines : 42     Critical/High lines : 3
------------------------------------------
==========================================
   OpenTrack — FINAL SUMMARY
==========================================
Project : OpenTrack     Version : 1.0     Lead : Ali Hassan
Contributors : 4   Names : ['Fatima', 'Jisoo', 'Minho', 'Sara']
Tech Stack   : {'Python', 'JavaScript', 'Rust', 'TypeScript'}
Issues : 5   Open : 3   Reporters : 4
Top Reporter : Minho (2 issues)
Critical:2  High:2  Medium:1
Report : open_track/project_report.txt
CSV    : open_track/issues.csv
==========================================
OpenTrack complete. Thank you for contributing to open source!
==========================================
```

**(10 points)**

No pseudocode or hints given. Figure out the implementation yourself.

- **Part 1:** Using a **single line of list comprehension**, build **urgent**, titles of all issues where priority is **'Critical'** or **'High'**. Print the list and its length.
- **Part 2:** Open **project_report.txt** in **append mode ('a')**, add an **URGENT ISSUES** section listing all titles from **urgent**. Use **readlines()** to print the last **6 lines** and confirm the append worked.

### Marks Distribution

| Section | Description | Pts |
|---|---|---|
| Sec 1 | Tuple (all methods), List, Dict (update/get/copy), input(), banner, sort, slicing, negative index | 25 |
| Sec 2 | List (counter/index change/slice), Set (all ops), Dict (build by loop, keys/values/items, pop, grouping, top without max()) | 30 |
| Sec 3 | os module, write mode, try/except IOError, CSV, read/readline/readlines, FileNotFoundError, f-strings only, no hardcoding | 35 |
| ★ | List comprehension with compound condition, append mode, readlines() last 6 lines | 10 |
| — | **Total** | **100** |

## Submission Rules

- Submit a single .ipynb or .py file to the course portal before the deadline
- All parts must be in **one single file** and do not submit separate files
- Code must run **without errors** from top to bottom and test before submitting
- Late penalty: **–10% per day**, maximum 3 days. No submissions after April 1, 2026
- **Plagiarism = zero marks** for all parties involved